USO · TUTORIAT 02 DIN 10

Text care curge prin țevi

Unde scrie un program, cum mutăm ce scrie în fișiere și cum legăm comenzi mici într-o conductă care rezolvă probleme mari.

```
student@fmi:~/tutoriat-02$ grep -i linux date.in | wc -l
```

PLANUL DE AZI

Cinci întrebări, apoi trecem la terminal

1

Unde scrie un program?

Cele trei fluxuri: `stdin`, `stdout`, `stderr`.

2

Cum trimit rezultatul într-un fișier?

Redirectări: `>`, `>>`, `<`, `2>`.

3

Cum leg două comenzi?

Pipe-ul `|` și filosofia UNIX.

4

Ce unelte am pentru text?

Filtre: `grep`, `sort`, `uniq`, `wc`, `cut`.

5

Când îmi „mănâncă” shell-ul caracterele?

Ghilimele și backslash.

Cuvinte pe care le vom folosi des. Apasă pe unul:

flux · descriptor · stdin · stdout · stderr · redirectare · pipe · filtru · tipar

flux (stream) Un șir de octeți care curge într-o singură direcție: dinspre o sursă (tastatură, fișier) spre o destinație (ecran, fișier, alt program).

Cum împărțim cele 90 de minute



1 · UNDE SCRIE UN PROGRAM

Orice program pornește cu trei „țevi” deja montate

Fiecare are un număr, numit descriptor. Apasă pe un număr ca să vezi la ce folosește.



1 · stdout, ieșirea standard

Aici scrie programul rezultatul, datele. Asta e ce poți trimite într-un fișier cu > sau mai departe printr-un pipe.

În C: stdout, printf(), puts(), write(1, ...)

```
$ ls date.in lipsa.txt
ls: cannot access 'lipsa.txt': No such file or directory
date.in
```

venit pe stdout venit pe stderr

2 · REDIRECTĂRI

Mută o țeavă: de pe ecran într-un fișier

Comanda e mereu aceeași: `ls date.in lipsa.txt` (primul fișier există, al doilea nu).
Alege ce adaugi la final.

(nimic) | > rez.txt | 2> err.txt | 2> /dev/null | > rez.txt 2>&1 | 2>&1 > rez.txt | &> tot.txt | | wc -l

```
$ ls date.in lipsa.txt > rez.txt
```

1. stdout → rez.txt
2. stderr → ecran

Doar stdout s-a mutat în fișier. Eroarea a rămas pe ecran, ca s-o vezi.

ecranul

```
ls: cannot access 'lipsa.txt': No such file or directory
```

rez.txt

```
date.in
```

> **golește fișierul întâi.** >> adaugă la final. Încearcă: `echo pas > f.txt` `echo pas >> f.txt`

f.txt

```
pas 1
pas 2
```

3 · CONDUCTE

Construiește o conductă, bucată cu bucată

Datele: `jurnal.txt`, 25 de conectări. Fiecare | dă ieșirea din stânga ca intrare celei din dreapta.

Câte încercări au eșuat? | Cine apare cel mai des? | Cine greșește parola? | Câte intrări pe zi? | Fără sort, uniq -c greșește

`| cut -d' ' -f3` `| cut -d' ' -f1` `| grep esuat` `| grep root` `| sort` `| uniq` `| uniq -c` `| sort -nr` `| head -n 3` `| wc -l`

```
cat jurnal.txt | cut -d' ' -f3 | sort | uniq -c | sort -nr | head -n 3
```

```
$ cat jurnal.txt | cut -d' ' -f3 | sort | uniq -c | sort -nr | head -n 3
      6 ana
      5 root
      4 diana
```

`cat jurnal.txt`	25 linii
`cut -d' ' -f3`	25 linii
`sort`	25 linii
`uniq -c`	6 linii
`sort -nr`	6 linii
`head -n 3`	3 linii

Scoate ultima | De la capăt

4 · FILTRE: GREP

Păstrează doar liniile care conțin un tipar

$ grep `-i` `-v` `-n` `-c` `linux` date.in

Intrarea (`date.in`). Verde = linia trece prin filtru.

```
1  Linux este un nucleu de sistem de operare scris in C.
2  Shell-ul citeste comenzile si porneste programele.
3  Comanda grep cauta un tipar in fiecare linie de text.
4  Comanda sort ordoneaza liniile, iar uniq elimina dublurile vecine.
5  Kernelul Linux gestioneaza procesele si memoria.
6  Un pipe leaga iesirea unei comenzi de intrarea urmatoarei.
7  Mesajele de eroare merg pe stderr, nu pe stdout.
8  Distributiile linux populare sunt Debian, Ubuntu si Fedora.
9  Comanda wc numara liniile, cuvintele si octetii.
10 Redirectarea > suprascrie fisierul, iar >> adauga la final.
11 In LINUX literele mari si mici conteaza.
12 Shell-ul face expandarea mastilor inainte sa porneasca grep.
```

Ce afișează `grep`:

```
$ grep linux date.in
Distributiile linux populare sunt Debian, Ubuntu
si Fedora.
$ echo $?
0  # a găsit ceva
```

^ la începutul tiparului = „linia începe cu”. Încearcă `^comanda` cu `-i`.

De ce scriem mereu `sort | uniq`?

`uniq` se uită doar la linia de dinainte. Dublurile care nu stau una lângă alta îi scapă.

`uniq` | **`uniq -c`** | `sort` | `sort | uniq` | `sort | uniq -c` | `sort -u`

intrarea

```
ana
bogdan
root
root
root
cristi
cristi
ana
diana
bogdan
```

→

după uniq -c

```
1 ana
1 bogdan
3 root
2 cristi
1 ana
1 diana
1 bogdan
```

Numere: `sort` vs `sort -n`

Fără `-n`, `sort` compară text, caracter cu caracter: `"10"` începe cu 1, deci vine înaintea lui `"9"`.

`sort` | `sort -n` | `sort -nr`

```
9
10
2
100
25
```

→

```
10
100
2
25
9
```

> Rețeta de reținut: `... | sort | uniq -c | sort -nr | head` = „cele mai dese valori, primele”.

5 · DESPECIALIZARE

Ce primește de fapt programul?

Înainte să pornească o comandă, shell-ul înlocuiește variabilele (`$HOME`), măștile (`*.c`) și taie textul la spații. Ghilimelele îl opresc.

```
echo $HOME *.c
```

```
echo "$HOME *.c"
```

```
echo '$HOME *.c'
```

```
echo \$HOME \*.c
```

```
grep ana login jurnal.txt
```

```
grep 'ana login' jurnal.txt
```

```
touch fisier nou.txt
```

Argumentele primite de program (`argv`), după ce a lucrat shell-ul:

`argv[0]` `echo` `argv[1]` `/home/student` `argv[2]` `exercitiu.c`

```
$ echo $HOME *.c
/home/student exercitiu.c
```

Fără ghilimele: $HOME devine calea, *.c devine lista de fișiere .c din folder (masca din Tutoriatul 01).

`"..."`	păstrează spațiile și *, dar **înlocuiește $VAR**
`'...'`	nu înlocuiește **nimic**: textul rămâne exact cum l-ai scris
`\`	oprește un **singur** caracter: **\$**, *****, **\ ** (spațiu)

EXERCIȚIUL DE AZI

Scriem în C un mini-`grep` care se poartă frumos

Patru reguli pentru un program bun de pus într-un pipe. Apoi rulează-l:

1. Citește din fișierul primit sau, dacă lipsește, din `stdin`.
2. Liniile găsite pe `stdout`. Erorile și statistica pe `stderr`.
3. Cod de ieșire ca la `grep`: `0` găsit, `1` negăsit, `2` eroare.
4. Închide cu `fclose()` doar ce a deschis cu `fopen()`.

```
./exercitiu -n Linux date.in
cat date.in | ./exercitiu -c -i linux
./exercitiu -i linux date.in > gasite.out
./exercitiu -v -i linux < date.in 2> /dev/null | wc -l
./exercitiu windows date.in
./exercitiu linux lipsa.txt
./exercitiu -q linux date.in
```

```
$ ./exercitiu -n Linux date.in
1:Linux este un nucleu de sistem de operare scris in C.
5:Kernelul Linux gestioneaza procesele si memoria.
[filtru] 2 din 12 linii potrivite (sursa: date.in)
$ echo $?
0
```

stdout stderr · fișier ca argument

FIȘĂ RAPIDĂ

Filtrele pe care le folosești azi

Apasă pe o comandă ca să vezi un exemplu pe `jurnal.txt`.

grep caută	**sort** ordonează	**uniq** fără dubluri	**wc** numără	**cut** taie coloane	**tr** înlocuiește caractere
head începutul	**tail** finalul	**tee** ramificație	**less** paginează	**diff** compară	**/dev/null** gaura neagră

```
$ cut -d' ' -f4 jurnal.txt | sort | uniq -c
     11 login
      5 logout
      5 ssh
      4 sudo
```

uniq

Elimină liniile identice vecine. -c le și numără. Aproape mereu după sort.

GREȘELI FRECVENTE

Capcanele clasice cu pipe-uri și redirectări

Apasă pe una: vezi de ce se întâmplă și cum o eviți.

```
sort date.txt > date.txt # fișierul rămâne gol!
```

DE CE

Shell-ul pregătește redirectarea înainte să pornească sort: deschide date.txt și îl golește. sort citește apoi un fișier gol.

REZOLVARE

```
sort date.txt > tmp.txt && mv tmp.txt date.txt
```

```
cut -d' ' -f3 jurnal.txt | uniq -c # ana apare pe 6 rânduri separate
```

```
make 2>&1 > log.txt # erorile tot pe ecran
```

```
grep: login: No such file or directory
```

```
sort numere.txt → 10 100 2 25 9
```

```
Erorile programului meu apar în fișierul cu rezultate
```

ÎNCĂLZIRE ÎNAINTE DE KAHOOT

Trei întrebări rapide

Fișierul f.txt are 10 linii. Câte are după echo gata > f.txt?

- 11
- 1
- 10

Pe ce descriptor trebuie să scrie un program mesajele de eroare?

- 0 (stdin)
- 1 (stdout)
- 2 (stderr)

Ce afișează echo '$HOME'?

- /home/student
- $HOME
- nimic

ACUM E RÂNDUL TĂU

Deschide terminalul și urmează pașii

Întâi exercițiile cu pipe-uri din `pipeline.sh`, apoi `TODO`-urile din scheletul `exercitiu.c`. Bifează pe măsură ce avansezi.

- [ ] `cd tutoriat-02/schelet` intră în folder
- [ ] `bash pipeline.sh` exercițiile 1–3, cu pipe-uri
- [ ] `nano exercitiu.c` rezolvă TODO 1–9
- [ ] `make && make test` compilează și testează
- [ ] `make pipeline` exercițiile 4–5, cu programul tău
- [ ] `make valgrind` bonus: nimic uitat deschis?

Ce exersezi

Citești din fișier *sau* din `stdin`, separi datele (`stdout`) de mesaje (`stderr`) și întorci un cod de ieșire cu care se pot face `&&` și `||`.

Bonus: Valgrind

Comentează `fclose(in)`, rulează `make valgrind` și citește ce ți-a rămas deschis. Îl folosim pe larg în Tutoriatul 04.

Te-ai blocat? Soluțiile sunt în `tutoriat-02/solutii/`. Încearcă întâi singur.